CIN - UNIVERSIDADE FEDERAL DE PERNAMBUCO (UFPE)
PTC CITi 26.2

RELATÓRIO

Folhagem & Cia · Desafio de back-end

Aspirante: David Gabriel Gomes Silva
E-mail: dggs2@cin.ufpe.br

Recife, 2026

1. O que foi feito:

- As 7 rotas obrigatórias e as 7 regras de negócio do enunciado.
- Bônus feitos: busca por nome, ordenação e a rota de plantas esgotadas.
- Ficou de fora: o script de seed, por falta de tempo.
- Extras: venda protegida contra compras simultâneas, resposta 400 em JSON quando o corpo vem com JSON mal formatado e limite para os números inteiros.

2. Como foi feito:

Ordem em que trabalhei:

1. Configurei o notebook.
2. Rodei e testei o CRUD de usuários do treinamento.
3. Segui os 7 passos do PDF, com um commit por passo.
4. Fiz uma revisão do código.
5. Fiz os bônus.
6. Escrevi o README.

Decisões e por quê:

- Usei === undefined em vez de !campo para saber se um campo foi enviado, porque o ! trata o 0 e o false como se fossem "nada".
- Fiz uma única função de validação para o cadastro e a edição, porque as regras de cada campo são iguais no cadastro e na edição (preço maior que zero, e assim por diante).
- Desestruturei o corpo da requisição em vez de mandar o req.body direto para o banco, porque ele traz tudo o que o cliente mandou, até o que não devia.
- Coloquei um limite nos números inteiros (2.147.483.647), porque um número maior passava na validação, mas o banco não aguenta mais que isso, aí quebrava no banco e virava 500.
- Fiz a venda com um updateMany que só diminui o estoque se estoque >= quantidade, porque tem o caso de duas compras lerem o estoque antes de subtrair e as duas passarem.
- Calculei o valor total com um for no código, porque o Prisma não sabe multiplicar preço × estoque de cada planta antes de somar. Por isso, a conta é feita no código. Com a loja vazia, o for não roda e os totais ficam em zero.
- Registrei a rota /plantas/esgotadas antes de /plantas/:id, porque o Express usa a primeira rota que combina com a URL.

Como testei: com um arquivo .http no REST Client, com uma requisição para cada caso da parte 4 do PDF, e com um teste de 5 vendas simultâneas usando o curl.exe.

Decisão diferente do enunciado: o PDF fala em entregar numa branch do repositório do treinamento, mas, conforme a orientação no Discord, criei meu próprio repositório pelo "Use this template", com o código na branch main. Criei também a branch desafio/david-gabriel, com o mesmo conteúdo da main.

3. Principais dificuldades

Sem dúvida, configurar o notebook para o desafio foi a dificuldade inicial. Comecei usando o Gemini, mas os comandos estavam confusos e não davam o resultado certo. Cheguei a fazer uma boa parte do desafio em outro repositório, mas estava dando muito erro e eu não estava conseguindo resolver. Optei por reiniciar o processo: apaguei as configurações do ambiente e recomecei com o Claude, tanto a instalação quanto uma nova cópia do template do repositório. Pelo menos a maioria dos códigos seguia o mesmo caminho e pude reutilizá-los, mas pedi para o Claude me falar o que estava errado e como reestruturar para seguir certo agora. Esse foi o principal obstáculo, ter que reiniciar o processo, mas valeu a pena, mesmo que não tenha dado tempo de fazer o último bônus.

Também tive alguns erros e dúvidas na parte técnica e nos testes, mas a maioria resolvi perguntando ao Claude o que estava acontecendo e seguindo o passo a passo para resolver. Pedi para ele listar os principais que resolvemos juntos para colocar aqui no relatório:

O que aconteceu	Como percebi	Como resolvi
npm bloqueado	Erro "a execução de scripts foi desabilitada"	Set-ExecutionPolicy RemoteSigned
Thunder Client passou a exigir login	Pediu uma conta ao enviar a primeira requisição	Troquei pelo REST Client
schema.prisma marcado em vermelho	Aviso da extensão do Prisma 7	"Pin to Prisma 6"
git add . incluiu arquivos pessoais	O git commit sem -m travou esperando o editor	Cancelei, usei git reset e o .git/info/exclude
500 ao cadastrar a primeira planta	Li os logs: Cannot convert undefined or null to object	O Prisma Client do container não tinha se atualizado: rodei yarn generate dentro do container e reiniciei

tsconfig.json com erros no editor	Número vermelho no Explorer	"Use Workspace Version" do TypeScript
JSON mal formatado respondia em HTML	Testando a requisição 36	Criei um tratador de erro no server.ts

4. Uso de inteligência artificial

Ferramentas: Claude, Copilot e, no começo, o Gemini.

Para quê:

- **Guiar a instalação e a configuração do ambiente.** Eu não tinha praticamente nenhum dos programas necessários para o desafio, só o VS Code. O Git não estava configurado porque troquei de notebook recentemente, e eu não tinha nenhuma experiência com as outras linguagens e ferramentas.
- **Formatação de textos.** Todos os textos (comentários, prints, o README e este relatório) tiveram a ortografia e a formalidade corrigidas pelo Claude, com o objetivo de acelerar o processo, formalizar a entrega e deixar tudo mais organizado. Os commits de correção de ortografia foram ele apontando onde eu tinha errado nos comentários, já que, por boa prática, é melhor deixá-los legíveis.
- **Explicar os conceitos.** Como eram termos muito técnicos e eu não estava familiarizado com todos, inicialmente pedi para ele me reexplicar o conteúdo dos treinamentos. Depois, enviei o desafio e perguntei o que eu precisava saber para realizá-lo, e ele foi me explicando cada parte. Em seguida, pedi o passo a passo para a resolução do desafio, num prompt que ia validando cada etapa: ele passava um passo, eu realizava, mandava o resultado para ele, perguntava se estava válido para fazer o commit e pedia os testes para realizar.
- **Diagnosticar erros a partir dos logs.** Conferir meus erros com essa estratégia me ajudou a ver onde errei e a extrair o máximo possível do código.
- **Fornecer o código.** O código de alguns passos vieram do Claude. Eu digitava em vez de copiar e colar (com a ajuda do autocompletar do Copilot), comentava do meu jeito para fixar o entendimento e testava. Quando não entendia alguma parte, pedia a explicação.
- **Propor os testes.** Como já citei, usei para gerar o arquivo .http, que eu ia conferindo e com o qual fiz os testes durante o desafio.
- **Fazer o rascunho do README.** Pedi como eu deveria estruturar meu README, já que ele tinha acesso ao meu projeto e sabia como eu o fiz. Então escrevi um rascunho no Google Docs mesmo (com erros e informal), depois pedi para o Claude melhorar a formalidade, a ortografia e colocar na estrutura de README para o GitHub. Também pedi para ele me falar o que eu podia adicionar para complementar.

Como conferi:

- rodei e testei cada passo (status e corpo das respostas);
- rodei o yarn typecheck;
- comparei com o CRUD de usuários e com o PDF.

O que aprendi: sinceramente, ainda não me acho capaz de realizar todas as etapas do desafio completamente sozinho. Mas foi extremamente útil para aprender conceitos, entender o direcionamento de cada código e a função que cada componente realiza no processo. Relembrei como usar o Git e o GitHub (já tinha perdido a prática desde o projeto final da disciplina de Introdução à Programação). E, mesmo que eu não tenha atingido uma ótima performance nas linguagens, foi ótimo ter uma base de lógica e sintaxe em cada uma, considerando que meu repertório até então era Python e C++ intermediário. Acredito que entender o que é e como funciona uma API seja um conhecimento útil e significativo para além do CITi, tanto para a vida profissional quanto para o meu desenvolvimento pessoal. Por isso, tentei fazer tudo da forma mais correta possível, mesmo com os gargalos e as dificuldades.